Matematiči fakultet
Univerzitet u Beogradu

Minimum sorting by reversals

Matija Đorđević 59/2022
Filip Nedeljković 43/2022

1. Uvod

1.1. Definicija problema

Neka je $n \in N$. Permutacija skupa $\{1, 2,..., n\}$ je bijetkivno preslikavanje $\pi: \{1, 2,..., n\} \rightarrow \{1, 2,..., n\}$ koje se zapisauje u obliku: $\pi: (\pi_1, \pi_2, ..., \pi_n)$ gde su svi elementi različiti i predstavlja raspored brojeva od 1 do n. Identična (sortirana) permutacija se označava kao id: $(1, 2, ..., n)$.

Za interval [i,j] obrtaj predstavlja permutaciju
$\pi: (1, 2, ..., i - 1, j, j - 1, ..., i + 1, i, j + 1, ..., n)$.

Problem minimalnog sortiranja obrtajima (Minimum sorting by reversals) za datu permutaciju π predstavlja pronalaženje najmanjeg niza obrtaja $\rho_1 ... \rho_n$ takvog da važi $\pi \bullet \rho_1 \bullet \rho_2 \bullet ... \bullet \rho_n = id$. Veličina tog niza obrataja se naziva obrtajna distanca i označava se sa $d(\pi)$

Neformalno neka je dat niz brojeva od 1 do n takav da se svaki broj pojavljuje tačno jednom. Potrebno je pronaći minimalni niz obrtaja takav da početni niz bude sortiran.

Ovaj problem je dokazan da pripada skupu **NP** teških problema od strane Alberta Kaprare (Alberto Caprara) u njegovom radu „Sortiranje obrtajima je teško" (Sorting by reversals is difficult) objavljenog 1997. godine.

Problem koji je sličan ovom je minimalno sortiranje obrtajima označenih permutacija. Za razliku od problema za neoznačene permutacije ovaj problem je dokazano rešiv u polinomijalnom vremenu od strane Pevznera, Bafina, Hanenhalija (Pavel A. Pevzner , Vineet Bafna, Sridhar Hannenhalli). U ovom radu bavičemo se problemom minimalnog sortiranja kod neoznačenih permutacija.

1.2 Primena

Koristi se u bioinformatici. Neophodan je da se odredi sličnost vrsta. Najmanji broj ukrštanja nam govori o tome kolika je razlika medju njima.

Jedna od najvažnijih primena problema minimalnog sortiranja obrtajima nalazi se u komparativnoj genomici.

Genom organizma može se modelovati kao permutacija gena. Tokom evolucije dolazi do mutacija koje menjaju raspored segmenata DNK. Jedan od tipova takvih mutacija jeste inverzija, pri kojoj se određeni segment genoma obrće.

Ako se genomi dve vrste modeluju kao permutacije istih gena, tada minimalan broj obrtaja potreban da se jedna permutacija transformiše u drugu predstavlja meru njihove evolutivne udaljenosti, tako da broj obrtaja postaje matematička mera sličnosti između organizama.

1.3. Postojeći naučni doprinosi

Problem minimalnog sortiranja obrtajima nastao je u kontekstu komparativne genomike, gde se genomi modeluju kao permutacije gena, a evolutivne promene predstavljaju operacije nad tim permutacijama. Jedan od pionira u formalizaciji ovakvih modela bio je David Sankof, koji je u svom radu objavljenog 1995. (Exact and approximation algorithms for sorting by reversal, with application to genome rearrangement) postavio temelje matematičkog pristupa analizi genomskih rearanžmana i definisanju metričkih mera udaljenosti između genoma.

Ključna prekretnica u proučavanju problema sortiranja obrtajima predstavlja rad Alberta Kaprare iz 1997. godine, u kome je dokazano da je problem minimalnog sortiranja obrtajima za neoznačene (unsigned) permutacije NP-težak. Ovaj rezultat ima značajne implikacije, jer ukazuje na to da se za opšti slučaj ne može očekivati postojanje efikasnog (polinomijalnog) algoritma za pronalaženje optimalnog rešenja. Nakon ovog rezultata, istraživanja su bila usmerena ka razvoju aproksimacionih algoritama i heuristika koje daju dobra, ali ne nužno optimalna rešenja.

Paralelno sa tim, intenzivno je proučavana i varijanta problema za označene (signed) permutacije. Značajan napredak postigli su Hanenhali i Pevzner ,koji su razvili polinomijalni algoritam zasnovan na analizi tzv. grafa tački prekida. Njihov rad je

pokazao da dodatna struktura (orijentacija elemenata) bitno menja računsku složenost problema.

Dalja istraživanja bavila su se unapređenjem aproksimacionih metoda za slučaj neoznačenih permutacija, analizom donjih i gornjih granica obrtajne distance, kao i eksperimentalnim ispitivanjem efikasnosti različitih heuristika. Problem i danas ostaje predmet interesovanja zbog svoje kombinacije teorijske složenosti i praktične primene u bioinformatici.

2. Algoritamski pristupi rešavanju problema

Jedna od ključnih ideja pri rešavanju problema minimalnog sortiranja obrtajima jeste analiza strukture permutacije pomoću tzv. **tačaka prekida (breakpoints)**. Ovaj koncept omogućava procenu koliko je trenutna permutacija udaljena od sortirane i koristi se za konstruisanje heuristika i ograničenja u algoritmima pretrage.

2.1 Tačke prekida

Tačka prekida predstavlja mesto u permutaciji gde se narušava očekivani redosled elemenata. Intuitivno, ona označava prekid monotonog segmenta permutacije.

Formalnije, neka je data permutacija π: $(\pi_1, \pi_2, \ldots, \pi_n)$. Proširimo permutaciju dodavanjem graničnih elemenata: π': $(0, \pi_1, \pi_2, \ldots, \pi_n, n+1)$. Par susednih elemenata (π'_i, π'_{i+1}) predstavlja **tačku prekida** ukoliko važi: $|\pi'_i - \pi'_{i+1}| \neq 1$.

Drugim rečima, ako dva uzastopna elementa ne predstavljaju uzastopne brojeve u prirodnom poretku, između njih postoji tačka prekida.

Na primer, permutacija π: $(2\ 3\ 1\ 5\ 6\ 4\ 8\ 7\ 9)$ sadrži tačke prekida između elemenata:

- 3 i 1
- 1 i 5
- 6 i 4
- 4 i 8
- 7 i 9

Sortirana permutacija $(1\ 2 \ldots n)$ nema nijednu tačku prekida. Stoga je prirodna strategija pokušati eliminisati tačke prekida primenom odgovarajućih obrtaja.

Važno svojstvo je da **jedna reverzija može ukloniti najviše dve tačke prekida**. Iz toga sledi donja granica za broj potrebnih operacija:

$d(\pi) \geq \lceil \frac{b(\pi)}{2} \rceil$ gde je $b(\pi)$ broj tačaka prekida

Ova procena predstavlja **heurističko donje ograničenje** koje se koristi u algoritmima pretrage.

2.2 Algoritam potpune pretrage (BFS)

Najjednostavniji način rešavanja problema jeste potpuna pretraga prostora svih mogućih permutacija korišćenjem pretrage u širinu.

Ideja algoritma je da se permutacija posmatra kao čvor u grafu čiji su susedi sve permutacije koje se mogu dobiti jednim obrtajem. BFS zatim istražuje ovaj graf po nivoima: prvo sve permutacije udaljene za 1 operaciju, zatim sve udaljene za 2 operacije, itd.

Pošto BFS obilazi čvorove po rastućoj dubini, prvi put kada se pronađe sortirana permutacija garantuje se da je pronađeno **optimalno rešenje**, odnosno minimalan broj reverzija.

Složenost.

Problem ovog pristupa je velika složenost izvršavanja. Broj mogućih permutacija iznosi $n!$, a broj mogućih obrtaja reda veličine $O(n^2)$ tako da je ukupna složenost izvršavanja $O(n^2 n!)$. Takođe je i prostorna složenost je veoma velika, reda $O(n!)$, jer je potrebno čuvati sve posećene permutacije. Čime postoje neprihvatljivo za velike ulaze n.

2.3 Heurističko gornje ograničenje (Pohlepni algoritam)

Kako bi se pretraga ubrzala, koristi se heuristika zasnovana na tačkama prekida. Ideja je konstruisati brzo, iako ne nužno optimalno, rešenje koje daje **gornju granicu** za optimalan broj obrtaja.

U implementaciji se koristi pohlepni (greedy) pristup. U svakom koraku razmatraju se svi mogući obrtaji i bira se ona koja najviše smanjuje broj tačaka prekida.

Maksimalno moguće poboljšanje u jednom koraku je za 2 tačke prekida, pa ako se pronađe takav obrtaj prekida se pretraga u tom koraku i taj obrtaj se ubacuje u niz.

Ovakav pristup ne garantuje minimalan broj operacija, ali obično daje relativno dobro rešenje koje služi kao gornja granica prilikom daljeg pretraživanja.

Dokaz da ne daje minimalan broj operacija sledi iz primera:

- „Greedy": $3412 \rightarrow 4312 \rightarrow 4321 \rightarrow 1234$
- Minimalan: $3412 \rightarrow 1432 \rightarrow 1234$

Odakle vidimo da je algoritam za gornje ograničenje uzima 3 koraka dok je minimalan broj koraka 2.

Složenost.

U jednom koraku razmatra se $O(n^2)$ mogućih reverzija, a za svaku se računa broj tačaka prekida što zahteva $O(n)$ vremena. Zbog toga je složenost jednog koraka $O(n^3)$. Kako se broj koraka u praksi retko približava n, ukupna složenost heuristike je približno $O(n^4)$, što je značajno brže od potpune pretrage.

2.4 Algoritam grananja i odsecanja

Da bi se prevazišla ograničenja potpune pretrage, koristimo **algoritam grananja i odsecanja** (branch and bound). Osnovna ideja ovog pristupa je sistematsko pretraživanje prostora rešenja uz korišćenje gornjih i donjih ograničenja kako bi se nepotrebne grane pretrage bile eliminisane.

Algoritam funkcioniše na sledeći način.

Najpre se pomoću pohlepne heuristike izračunava gornja granica za broj obrtaja potrebnih za sortiranje permutacije. Ova vrednost predstavlja trenutno najbolje poznato rešenje.

Zatim se vrši pretraga prostora permutacija generisanjem svih mogućih obrtaja nad trenutnom permutacijom. Za svaku novu permutaciju računa se broj tačaka prekida i odgovarajuće donje ograničenje $LB = \lceil \frac{b(\pi)}{2} \rceil$.

Ako važi: $trenuta_dubina + LB \geq gornja_granica$

tada je jasno da ta grana pretrage ne može dovesti do boljeg rešenja od već pronađenog. U tom slučaju pretraga se **prekida (pruning)** i algoritam se ne spušta dalje niz tu granu.

Na ovaj način se značajno smanjuje broj permutacija koje je potrebno razmotriti u odnosu na potpunu pretragu.

Dodatna optimizacija u implementaciji jeste sortiranje kandidata prema broju preostalih tačaka prekida. Permutacije koje imaju manji broj tačaka prekida razmatraju se pre ostalih, jer je veća verovatnoća da će brže dovesti do dobrog rešenja i time dodatno smanjiti gornju granicu.

Rezultat algoritma je minimalan broj obrtaja pronađen tokom pretrage.

Složenost.

U teorijskom najgorem slučaju branch and bound i dalje može obići veliki deo prostora permutacija, pa njegova vremenska složenost ostaje eksponencijalna. Međutim, zahvaljujući donjoj granici zasnovanoj na broju tačaka prekida i gornjoj granici dobijenoj heuristikom, u praksi se veliki broj grana pretrage eliminiše. Zbog toga je algoritam znatno efikasniji od potpune pretrage i omogućava rešavanje većih instanci problema.

2.5 Funkcija cilja

Naredni algoritmi koji su korišćeni za reševanje problema jesu metoda promenljivih okolina i genetski algoritam. Za oba algoritma se za evaluaciju koristi ista funkcija cilja.

Prilikom izrade projekta testirano je više funkcija cilja. S obzirom da je naš cilj da pronađemo najmanji niz obrtaja koji sortira početnu permutaciju, prirodno se nameće da funkcija cilja bude dužina niza obrtaja. Međutim tada se ne vodi računa da li niz obrtaja zapravo izvršava sortiranje početne permutacije, zbog čega je pored dužine niza obrtaja potrebno razmatrati da li taj niz približava permutaciju ka sortiranoj. Zato je potrebno meriti kvalitet permutacije koja se dobija primenom niza obrtaja na početnu, a za to ćemo koristiti broj tačaka prekida u dobijenoj permutaciji.

Funkcija cilja koja je korićena na kraju jeste:
$fitness = \ -100 * number\ of\ breakpoints\ - |reversals|$
Funkcija cilja je negativna, jer se u algoritmima vrši maksimizacija funkcije. Kako bi se osiguralo da se algoritam navodi ka ispravnom rešenju, broju tačaka prekida je data velika težina. Time algoritmi prioritizuju prvensvrno rešenja koja uspešno sortirau početnu permutaciju.

Računanje tačaka prekida je linearne složenosti u odnosu na veličinu permutacije jer je potrebno jednom proći kroz celu permutaciju. Nalaženje dužine liste obrtaja u pajtonu je konstantne složenosti, pa je ukupna vremenska složenost evaluacije rešenja O(n).

2.6 Metoda promenljivih okolina

Metoda promenljivih okolina (Variable Neighborhood Search – VNS) predstavlja jedan od najzastupljenijih metaheuristički pristupa optimizaciji.

Na početku se generiše početno rešenje pokretanjem kratke lokalne pretrage. Lokalna pretraga ima nasumično biranje u sebi pa je i ovo rešenje nasumično, ali malo boljeg kvaliteta nego potpuno nasumično generisanje niza obrtaja.

Algoritam se određen broj iteracija izvršava. U svakoj iteraciji izvršavaju se operacije razmrdavanja rešenja (shaking) i lokalne pretrage nad novim rešenjem. Ove operacije se ponavljaju dok se ne pronađe novo najbolje rešenje ili je shaking izvršen određen broj puta k, zadat kao parametar funkcije, nakon čega kreće nova iteracija.

Shaking
Shaking funkcija prima trenutno najbolje rešenje i veličinu okoline(k).
Potom se k puta izvršava razmrdavanje koje može da bude ili dodavanje novog obrtaja na kraj, ili uklanjanje jednog obrtaja iz niza. Testiranjem su pokazani bolji rezultati da se da veća verovatnoća da se izvrši uklanjanje jednog obrtaja, a manja dodavanju novog.

Lokalna pretraga

Implementirana je jednostavna metoda lokalne pretrage. Ideja je da se nakon generisanja početnog rešenja razmatraju obrtaji koji smanjuju broj tačaka prekida i nasumično bira jedna od njih.

Nasumičan izbor poteza pomaže u izbegavanju lokalnih minimuma, odnosno situacija u kojima deterministička heuristika ne može dodatno poboljšati rešenje. Lokalna pretraga se izvršava ograničen broj koraka ili dok god dolazi do poboljšanja rešenja.

Složenost

složenost zavisi od broja iteracija I, maksimalne veličine okoline kmax, dužine permutacije n i broja obrtaja R. Tako da je složenost $O(I \cdot kmax \cdot n^3 \cdot R)$.

A ako se uključi opcija za lokalnu pretragu složenost postaje
$O(I \cdot kmax \cdot n^3 \cdot R)$.

2.7 Genetski algoritam

Genetski algoritam spada u p-metaheuristike, odnosno posmatra se veći broj rešenja istovremeno. Algoritam se zasniva na principima prirodne selekcije i evolucije.

Na početku se nasumično generiše populacija. Svaka jedinka je predstavljena kao lista obrtaja. Na početku ta lista ima proizvoljnu dužinu od 1 do gornje granice koja je zapravo rezultat izvršavanja pohlepnog algoritma. Nakon toga određen broj puta (broj generacija) izvršavaju redom selekcija ukrštanje i mutacija.

Pre selekcije vrši se izvajanje određenog broja najboljih jedinki koje direktno ulaze u sledeću generaciju(elitizam). Nad ostatkom populacije se vrši selekcija. U ovom radu je korišćena turnirska selekcija, tj. od određenog broja jedinki bira se ona sa najboljom prilagođenošću. Prilagođenost se meri prethodno navedenom funkcijom cilja. Veličina turnira se ze zadaje kao parametar algoritma

Nakon selekcije dva roditelja vrši se ukrštanje, koje je implementirano kao jednopoziciono ukrštanje. Razmatrana su bila i višepoziciona i uniformna ukrštanja, ali su ona dala gore rezultate nego jednopoziciono.

Nakon kreiranja deteta postoji mala verovatnoća, zadata kao parametar algoritma, da se izvrši mutacija. Mutacija može da bude ili dodavanje nasumičnog obrtaja na kraj, ili uklanjanje nasumičnog obrtaja iz liste.

Nakon mutacije moguće je izvršiti i kratku lokalnu pretragu nad detetom. Time ovaj algoritam postaje mimetički. Testiranjem je pokazano da bez lokalne pretrage algoritam daje loše rezultate čak i na malim permutacijama, tako da je na kraju lokalna pretraga uvek bila iskorišćena.

Složenost

Vremenska složenost implementiranog genetskog algoritma zavisi od veličine populacije P, broja generacija G, dužine permutacije nnn i broja obrtaja u jednom

rešenju R. Tako da je vremenska složenost bez lokalne pretrage $O(P \cdot G \cdot n \cdot R)$, dok je sa lokalnom pretragom složenost $O(P \cdot G \cdot n^3 \cdot R)$.

3. Rezultati

Testiranje je vršeno na nasumično generisanim podacima, zbog manja postojećih javno dostupnih podataka za testiranje. Test istance su podeljene u 3 grupe: male, srednje i velike. Broj i veličine instanci u grupama su date u tabeli.

Veličine instanci	Broj instanci	Dužina instanci
male	20	[5, 8]
srednje	15	[10, 15]
velike	10	[25, 30]

Test instance se generišu tako što se nasumično izabere veličina permutacije n u zadatim granicama, zatim se napravi skup brojeva od jedan do n. Iz skupa se uzima jedan po jedan broj dok se ne isprazni.

Ekperimentalno okruženje

- Operativni sistem: CachyOS x86_64
- Procesor: Intel(R) Pentium(R) Gold 7505 (4) @ 3.50 GHz
- Radna Memorija: 8GB
- Python verzija: 3.13.17

3.1. Parametri heuristika

U zavisnosti nad kojom grupom testova, parametri genetskog algoritma i metide promenljivih okolina su imali vrednosti zadate u tabelama.

Genetski algoritam	Veličina populacije	Broj generacija	Verovatnoća mutacije	Procenat elitizma	Veličina turnira
male	200	100	0.1	0.1	10
srednje	200	200	0.1	0.1	10
velike	200	400	0.1	0.15	10

VNS	Broj iteracija	Najmanja okolina	Najveća okolina
male	1000	1	5
srednje	2000	1	10
velike	5000	1	10

3.2. Diskuzija rezultata:

Metrike koje su korišćene za testiranje nad malim instacama su:

- average_time - prosečno vreme izvršavanja
- percent_of_optimal - procenat optimalnih rešenja; za optimalno rešenje se smetra rešenje dobijeno metodom grube sile
- percent_succesfully_sorted - pricenat rešenja koja ispravno sortiraju test instancu

Rezultati za male test instace su dati u tabeli.

	average_time	percent_of_optimal	percent_succesfully_sorted
bruteforce	0.072057	100.0	100.0
branch_and_bound	0.000481	100.0	100.0
greedy	0.000117	85.0	100.0
vns	0.241404	100.0	100.0
genetic_algorithm	1.053595	100.0	100.0

Nad srednjim test instancama korišćene su iste metrike. Za razliku od prethodnog testiranje, ovde se ne testira metoda grube sile jer je vremenski previše zahtevna, pa se

računanje metrike percent_of_optimal vrši u odnosu na rešenje dobijeno metodom grananja i odsecanja

	average_time	percent_of_optimal	percent_succesfully_sorted
branch_and_bound	3.954378	100.000000	100.0
greedy	0.000971	60.000000	100.0
vns	6.077249	60.000000	100.0
genetic_algorithm	7.871341	93.333333	100.0

Za velike test instance metoda grananja i odsecanja je takođe previše vremenski zahtevna, pa su korišćene samo pohlepi algoritam, genetski algoritam i algoritam promenljivih okolina. Pošto sada nemamo algoritam koji garantuje optimalno rešenje, umesto metrike percent_of_optimal se koristi metrika percent_best_result - procenat instanci u kome je određena metoda imala najmanje ispravno rešenje

	average_time	percent_best_result	percent_succesfully_sorted
greedy	0.021309	80.0	100.0
vns	146.774267	0.0	100.0
genetic_algorithm	126.932546	30.0	100.0

Za sve 3 grupe testova svi algoritmi vraćaju rešenja koja uspešno sortiraju instance.

Za male instance možemo primetiti da pohlepni pristup ne vraća uvek optimalno rešenje, tako da je on za ovu dimenziju problema loš izbor. Svi ostali algoritmi daju optimalno rešenje. Čak i za ovako male slučajeve, metoda grube sile je primetno sporija od od metode grananja i odsecanja, tako da bi za bilo koji slučaj trebalo izbegavati korišćenje te metode. Genetski algoritam i metoda promenljivih okolina su primetno sporije od ostalih algoritama, a ne garantuju optimalno rešenje, tako da i njih ne bi trebalo koristiti.

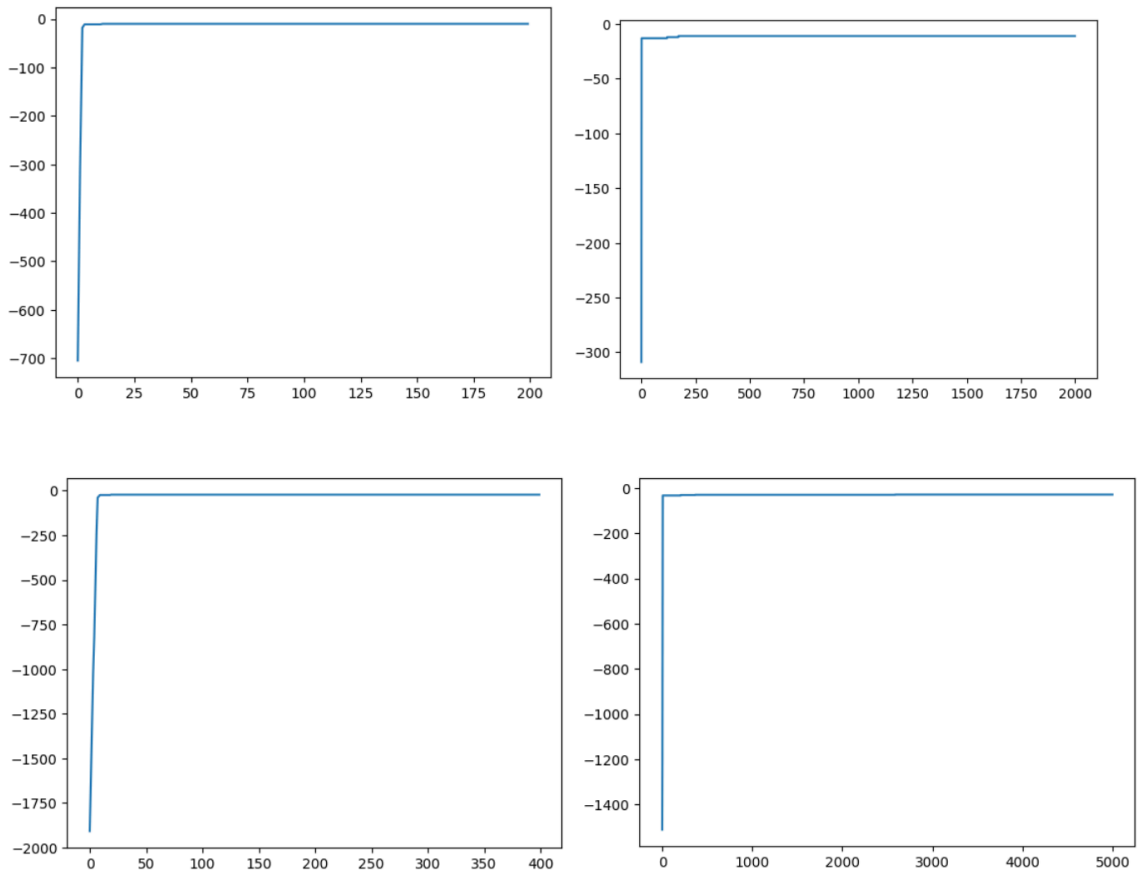
Za srednje instance metoda promenljivih okolina i pohlepni algoritmi imaju primetno lošiju optimalnost. Međutim, pohlepna metoda se izvršava daleko brže od ostalih algoritama, tako da ako su nam bitnije performanse nego preciznost, nju je moguće uzeti u obzir. Genetski algoritam nema savršenu optimalnost, ali je ona i dalje veoma visoka. U prosečnom vremenu on se jeste izvršavao sporije od metode grananja i odsecanja, ali ovde izvršavao samo 2 puta sporije, dok se za male instance izvršavao

sporije više hiljada puta. Tako da i minimalnim uvećanjem veličina instanci, ili čak testiranjem nad instancama iste veličine ali sa većom obrtajnom distancom, genetski algoritam će se potencijalno brže izvršavati.

Za velike instance vidimo da metoda promenljivih okolina se izvršava najsporije i nijednom nije dala najbolji rezultat. Pohlepni algoritam se izvršava daleko najbrže i najčešće ima najbolju vrednost. Naizgled genetski algoritam je gori od pohlepnog. Ali postoje slučajevi kada je genetski algoritam uspeo da vrati bolje rezultate od pohlepnog. Tako da ako nam je veoma bitno da nađemo što manji mogući niz obrtaja, možemo isprobati genetski algoritam u slučaju da vrati bolje rešenje. Genetskom algoritmu možemo dodatno menjati parametre tako da verovatnije daje bolje rezultate, Takođe, za razliku od pohlepnog algoritma koji uvek vraća istu vrednost, genetski algoritam možemo pokrenuti više puta i uzeti minimum rezultata svakog pokretanja.

3.3. Analiza promene funkcije cilja

Na graficima možemo videti premene funkcije u toku izvršavanja za genteski algoritam(levi grafici) i metodu promenljivih okolina (desni grafici). gornji grafici se odnose na srednju test instancu 10, a donji se odnose na veliku test instancu 1.



U oba slučaja za obe funkcije možemo videti drastičo približavanje funkcije cilje nuli. Na osnovu toga možemo da zaključimo da veoma brzo algoritmi pronalaze niz obrtaja koji ispravno sortira permutaciju. Time je broj tačaka prekida jednak nuli, pa na funkciju cilja utiče samo dužina niza obrtaja.

4. Zaključak

U ovom radu razmatran je problem minimalnog sortiranja obrtajima za neoznačene permutacije. Ovaj problem ima značajan teorijski i praktični značaj, naročito u oblasti bioinformatike, gde se koristi za procenu sličnosti između genoma različitih vrsta. S obzirom na to da je problem dokazano NP-težak, pronalaženje optimalnog rešenja za veće instance postaje računski zahtevno, zbog čega se često koriste heuristički i metaheuristički pristupi.

U okviru projekta implementirana su nekoliko takvih pristupa: bfs, pohlepni algoritam, algoritam grananja i odsecanja, genetski algoritam i algoritam promenljive okoline (VNS). Genetski algoritam koristi principe evolucije, kao što su selekcija, ukrštanje i mutacija, kako bi kroz generacije poboljšavao populaciju rešenja. Sa druge strane, metoda promenljive okoline se zasniva na sistematskoj promeni okoline rešenja i primeni lokalne pretrage kako bi se izbeglo zaglavljivanje u lokalnim optimumima.

Rezultati pokazuju da je za male i srednje instance najbolje koristiti metodu grananja i odsecanja, pohlepan algoritam je koristan za veće probleme, jer ima veoma dobre performanse i dalje rešenje pristojnog kvaliteta. Genetski algoritam može biti koristan za velike instance ako je potrebna najveća moguća tačnost, jer on ume ponekad da donese bolje rezultate nego pohlepni.

5. Literatura

[1] Sajt predmeta Računarska inteligencija, Stefan Kapunac, Matematički fakultet, Univerzitet u Beogradu. https://poincare.matf.bg.ac.rs/~stefan.kapunac/ri.html

[2] https://www.americanscientist.org/article/sorting-out-the-genome

[3] Pierluigi Crescenzi and Viggo Kann, A compendium of NP optimization problems, https://www.csc.kth.se/~viggo/wwwcompendium/node266.html

[4] Alberto Caprara, Sorting by reversals is difficult (1997)

[5] Johas Hubotter, On sorting by reversals (2020), https://jonhue.github.io/min-sbr/paper.pdf